# USBD38: Add a new gauge (bitola) - Sprint 3

Este documento descreve a implementação, configuração de dados e testes realizados para a **USBD38**.

## 1. Descrição da User Story

- As a Planner, I want to add a new gauge (bitola)

**Critérios de Aceitação e Regras de Negócio:**

- A bitola é identificada por um valor de largura (em mm) e um nome.
- O sistema deve validar se a largura é um valor positivo (> 0).
- O sistema deve impedir a duplicação de bitolas (mesma largura).
- O sistema deve impedir a duplicação de nomes, ignorando diferenças de maiúsculas/minúsculas e espaços extra (normalização).
- A implementação deve ser feita através de uma função PL/SQL.

---

## 2. Explicação do Script (Função PL/SQL)

A lógica central da **USBD38** foi encapsulada na função PL/SQL `insert_gauge`. Esta abordagem permite centralizar as regras de negócio e garantir a integridade dos dados, independentemente de onde a inserção é solicitada.

**Detalhes da Implementação:**

1. **Assinatura da Função:** A função aceita dois parâmetros: `p_gauge_name` (Nome) e `p_width_mm` (Largura). Retorna um número (`NUMBER`) que representa o código de estado da operação.
2. **Validação de Dados Obrigatórios:** Verifica se algum dos parâmetros é `NULL`. Retorna `0` em caso afirmativo.
3. **Validação de Regra de Negócio (Largura):** Impede a criação de bitolas com largura negativa ou zero. Retorna `-2` se inválido.
4. **Normalização de Texto:** Para garantir que a verificação de duplicados é robusta, o nome de entrada é normalizado:
    - Convertido para maiúsculas (`UPPER`).
    - Espaços nas extremidades removidos (`TRIM`).
    - Múltiplos espaços internos substituídos por um único espaço (`REGEXP_REPLACE`).
5. **Verificação de Duplicados (Descrição):** Consulta a tabela `Railway_Gauge` comparando a descrição normalizada. Se já existir, retorna `-3`.
6. **Inserção Segura:** Executa o `INSERT`. Caso a chave primária (largura) já exista, o banco de dados lança a exceção `DUP_VAL_ON_INDEX`, que é capturada e traduzida para o código de retorno `-1`.
7. **Transação:** Realiza `COMMIT` em caso de sucesso ou `ROLLBACK` em caso de erro.

### Código: `usbd38-script.sql`

```
-- usbd38-script.sql
-- ------------------------------------------------------------
-- USBD38: Function to add a new gauge (Bitola)
-- ------------------------------------------------------------

CREATE OR REPLACE FUNCTION insert_gauge (
    p_gauge_name IN VARCHAR2,
    p_width_mm   IN NUMBER
)
RETURN NUMBER
IS
    v_count NUMBER;
    v_normalized_input VARCHAR2(200);
BEGIN
    -- 1. Validação: Campos obrigatórios
    IF p_gauge_name IS NULL OR p_width_mm IS NULL THEN
        RETURN 0;
END IF;

    -- 2. Regra de Negócio: Verificar se a largura é um número positivo
    IF p_width_mm <= 0 THEN
        RETURN -2;
END IF;

    -- 3. NORMALIZAÇÃO (Evitar a rejeição")
    -- Remove espaços no início/fim e transforma múltiplos espaços internos num só
    v_normalized_input := UPPER(TRIM(REGEXP_REPLACE(p_gauge_name, ' +', ' ')));

    -- 4. Regra de Negócio: Verificar se o nome já existe (com normalização rigorosa)
SELECT COUNT(*) INTO v_count
FROM Railway_Gauge
WHERE UPPER(TRIM(REGEXP_REPLACE(description, ' +', ' '))) = v_normalized_input;

IF v_count > 0 THEN
        RETURN -3; -- Nome duplicado
END IF;

    -- 5. Inserir a nova bitola
INSERT INTO Railway_Gauge (gauge_value, description)
VALUES (p_width_mm, TRIM(p_gauge_name));

COMMIT;
RETURN 1;

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        ROLLBACK;
RETURN -1; -- Valor de bitola (PK) duplicado
WHEN OTHERS THEN
        ROLLBACK;
RETURN 0;
END insert_gauge;
/
```

# 3. Explicação de Cada Teste

Para validar a User Story USBD38, foram desenvolvidos 5 cenários de teste que cobrem inserções válidas, violações de integridade e regras de normalização de dados.

### Teste 1: Inserção Bem-Sucedida

**Objetivo:** Verificar se é possível inserir uma nova bitola válida que ainda não existe no sistema. **Descrição:** O teste garante primeiro que o registo não existe e depois tenta inserir a "Bitola Métrica" com 1000mm. **Resultado Esperado:** A função deve retornar `1` e o registo deve ficar persistido na tabela.

```
 -- usbd38_test1.sql
SET SERVEROUTPUT ON;

DECLARE
    v_result NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('========================================');
    DBMS_OUTPUT.PUT_LINE('TESTE 1: INSERÇÃO BEM-SUCEDIDA');
    DBMS_OUTPUT.PUT_LINE('========================================');

    -- Setup: Garantir que não existe conflito prévio
    DELETE FROM Railway_Gauge WHERE gauge_value = 1000;
    DELETE FROM Railway_Gauge WHERE UPPER(TRIM(description)) = 'BITOLA MÉTRICA';
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('A tentar inserir: Bitola Métrica (1000mm)...');

    v_result := insert_gauge('Bitola Métrica', 1000);

    DBMS_OUTPUT.PUT_LINE('Resultado obtido: ' || v_result || ' (Esperado: 1)');

    IF v_result = 1 THEN
        DBMS_OUTPUT.PUT_LINE('STATUS: [SUCESSO]');
    ELSE
        DBMS_OUTPUT.PUT_LINE('STATUS: [FALHA]');
    END IF;
    DBMS_OUTPUT.PUT_LINE('----------------------------------------');
END;
/

-- Validação Visual
SELECT * FROM Railway_Gauge WHERE gauge_value = 1000;
```

## Teste 2: Validações de Erros

**Objetivo:** Verificar se a função impede a inserção de dados inválidos ou que violem as regras de negócio. **Cenários:**

1. **Largura Duplicada:** Tentar inserir uma bitola com uma largura que já existe (ex: 1668mm).
2. **Largura Negativa:** Tentar inserir um valor negativo (-100mm).
3. **Largura Zero:** Tentar inserir valor 0.
4. **Nulos:** Tentar passar `NULL` nos parâmetros. **Resultado Esperado:**

- Largura Duplicada: `-1`
- Largura Inválida (<=0): `-2`
- Nulos: `0`

```
-- usbd38_test2.sql
SET SERVEROUTPUT ON;

DECLARE
    v_result NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('========================================');
    DBMS_OUTPUT.PUT_LINE('TESTE 2: VALIDAÇÕES DE ERRO');
    DBMS_OUTPUT.PUT_LINE('========================================');

    -- 2a: Largura duplicada (Ibérica 1668mm já existe no setup base)
    v_result := insert_gauge('Outra Ibérica', 1668);
    DBMS_OUTPUT.PUT_LINE('Teste PK Duplicada (1668mm): ' || v_result || ' (Esperado: -1)');

    -- 2b: Largura negativa
    v_result := insert_gauge('Bitola Negativa', -100);
    DBMS_OUTPUT.PUT_LINE('Teste Negativo (-100mm):     ' || v_result || ' (Esperado: -2)');

    -- 2c: Largura zero
    v_result := insert_gauge('Bitola Zero', 0);
    DBMS_OUTPUT.PUT_LINE('Teste Zero (0mm):            ' || v_result || ' (Esperado: -2)');

    -- 2d: Parâmetros Nulos
    v_result := insert_gauge(NULL, NULL);
    DBMS_OUTPUT.PUT_LINE('Teste Nulos:                 ' || v_result || ' (Esperado: 0)');

    DBMS_OUTPUT.PUT_LINE('----------------------------------------');
END;
/
```

## Teste 3: Validação de Nomes (Duplicados)

**Objetivo:** Garantir que não são criadas bitolas com a mesma descrição, ignorando a diferença entre maiúsculas e minúsculas ( *Case Insensitive*). **Descrição:** O teste tenta inserir "BITOLA MÉTRICA" (tudo em maiúsculas). Como a "Bitola Métrica" (misto) já foi inserida com sucesso no Teste 1, o sistema deve detetar a semelhança semântica e bloquear a inserção. **Resultado Esperado:** Retorno `-3` (Nome duplicado).

```
-- usbd38_test3.sql
SET SERVEROUTPUT ON;

DECLARE
    v_result NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('========================================');
    DBMS_OUTPUT.PUT_LINE('TESTE 3: NOMES DUPLICADOS (CASE INSENSITIVE)');
    DBMS_OUTPUT.PUT_LINE('========================================');

    -- Tentar inserir nome igual ao do Teste 1 mas com Case diferente.
    -- O ID 1001 é usado para garantir que o erro não é de PK (1000), mas sim de Nome.
    DBMS_OUTPUT.PUT_LINE('Tentativa: inserir "BITOLA MÉTRICA" (Upper)...');

    v_result := insert_gauge('BITOLA MÉTRICA', 1001);

    DBMS_OUTPUT.PUT_LINE('Resultado: ' || v_result || ' (Esperado: -3)');

    IF v_result = -3 THEN
        DBMS_OUTPUT.PUT_LINE('STATUS: [SUCESSO] - Duplicado detetado corretamente.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('STATUS: [FALHA] - Permitiu duplicado ou deu erro incorreto.');
    END IF;
    DBMS_OUTPUT.PUT_LINE('----------------------------------------');
END;
/
```

## Teste 4: Inserções Múltiplas

**Objetivo:** Validar a robustez e estabilidade do sistema ao processar várias transações válidas sequencialmente. **Descrição:** O teste insere três bitolas válidas diferentes (Russa, Estreita, Indiana) uma após a outra para garantir que a função lida corretamente com múltiplos pedidos sem erros de estado ou bloqueios. **Resultado Esperado:** Todas as operações devem retornar `1` (Sucesso) e os dados devem persistir na base de dados.

```
-- usbd38_test4.sql
SET SERVEROUTPUT ON;

DECLARE
    v_r1 NUMBER;
    v_r2 NUMBER;
    v_r3 NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('========================================');
    DBMS_OUTPUT.PUT_LINE('TESTE 4: INSERÇÕES MÚLTIPLAS');
    DBMS_OUTPUT.PUT_LINE('========================================');

    -- Limpeza preventiva para garantir que o teste corre bem
    DELETE FROM Railway_Gauge WHERE gauge_value IN (1520, 600, 1676);
    COMMIT;

    -- Inserção 1: Bitola Russa
    v_r1 := insert_gauge('Bitola Russa', 1520);
    DBMS_OUTPUT.PUT_LINE('Bitola Russa (1520):   ' || v_r1 || ' (Esperado: 1)');

    -- Inserção 2: Bitola Estreita
    v_r2 := insert_gauge('Bitola Estreita', 600);
    DBMS_OUTPUT.PUT_LINE('Bitola Estreita (600): ' || v_r2 || ' (Esperado: 1)');

    -- Inserção 3: Bitola Indiana
    v_r3 := insert_gauge('Bitola Indiana', 1676);
    DBMS_OUTPUT.PUT_LINE('Bitola Indiana (1676): ' || v_r3 || ' (Esperado: 1)');

    IF v_r1 = 1 AND v_r2 = 1 AND v_r3 = 1 THEN
        DBMS_OUTPUT.PUT_LINE('STATUS: [SUCESSO] - Todas as inserções foram aceites.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('STATUS: [FALHA] - Uma ou mais inserções falharam.');
    END IF;
    DBMS_OUTPUT.PUT_LINE('----------------------------------------');
END;
/

-- Validação Visual: Verificar se os 3 registos existem
SELECT * FROM Railway_Gauge WHERE gauge_value IN (1520, 600, 1676);
```

## Teste 5: Validação Avançada de Normalização

**Objetivo:** Testar os limites da deteção de duplicados de nome, garantindo que o sistema é resiliente espaços extra, formatação inconsistente, e que normaliza corretamente antes de validar.

**Descrição:**

1. Limpa dados de teste (IDs 5000 a 5002).
2. Insere a "Bitola Europeia" (ID 5000) como referência limpa.
3. Tenta inserir "Bitola Europeia" (vários espaços internos, ID 5001).
4. Tenta inserir " bitola EUROPEIA " (espaços nas pontas e caixa misturada, ID 5002).

**Resultado Esperado:** O sistema deve normalizar internamente todas estas strings para "BITOLA EUROPEIA". Como o registo base já existe, as tentativas 3 e 4 devem ser rejeitadas como duplicados, retornando o código `-3`.

```
 -- usbd38_test5.sql
SET SERVEROUTPUT ON;

DECLARE
    v_result NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('========================================');
    DBMS_OUTPUT.PUT_LINE('TESTE 5: VALIDAÇÃO DE NORMALIZAÇÃO DE STRING');
    DBMS_OUTPUT.PUT_LINE('========================================');

    -- Setup: Limpeza de dados para o teste 5
    -- Usamos IDs fictícios (5000+) para não afetar dados reais de negócio
    DELETE FROM Railway_Gauge WHERE gauge_value IN (5000, 5001, 5002);
    COMMIT;

    -- 1. Inserção base para comparação
    DBMS_OUTPUT.PUT_LINE('1. Inserir Base: "Bitola Europeia" (5000mm)...');
    v_result := insert_gauge('Bitola Europeia', 5000);
    DBMS_OUTPUT.PUT_LINE('Resultado: ' || v_result || ' (Esperado: 1)');

    -- 2. Teste com múltiplos espaços internos
    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('2. Tentar inserir "Bitola    Europeia" (5001mm)...');
    v_result := insert_gauge('Bitola    Europeia', 5001);
    DBMS_OUTPUT.PUT_LINE('Resultado: ' || v_result || ' (Esperado: -3)');

    IF v_result = -3 THEN
        DBMS_OUTPUT.PUT_LINE('   -> Detetou duplicado com espaços internos.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('   -> FALHOU na normalização de espaços internos.');
    END IF;

    -- 3. Teste com espaços nas pontas e Case misturado
    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('3. Tentar inserir "  bitola EUROPEIA  " (5002mm)...');
    v_result := insert_gauge('  bitola EUROPEIA  ', 5002);
    DBMS_OUTPUT.PUT_LINE('Resultado: ' || v_result || ' (Esperado: -3)');

    IF v_result = -3 THEN
        DBMS_OUTPUT.PUT_LINE('   -> Detetou duplicado com trim e upper case.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('   -> FALHOU na normalização de trim/case.');
    END IF;

    DBMS_OUTPUT.PUT_LINE('----------------------------------------');
END;
/

-- Validação Visual Final: Deve existir apenas o registo 5000
SELECT gauge_value, description FROM Railway_Gauge WHERE gauge_value BETWEEN 5000 AND 5002;
```